

Assignment A4: Interrupts and Timers

Andres Benitez & Luis Woo

a. CCR value calculations from Part A

Clock timer:

$$f_{\text{TIM2}} = 4 \text{ MHz}$$

A single clock tick:

$$T_{\text{tick}} = \frac{1}{\text{input clock}} = \frac{1}{4,000,000} = 0.25 \mu\text{s}$$

Desired output:

$$f_{\text{out}} = 5 \text{ kHz} \quad \text{Period: } \frac{1}{5000} = 200 \mu\text{s}$$
$$\frac{\text{Period}}{\text{Single clock tick}} = \frac{200 \mu\text{s}}{0.25 \mu\text{s}} = 800 \text{ ticks} \quad \boxed{\text{ARR} = 799} \text{ Note: [0, 799]}$$

Auto-Reload Register (ARR)

- Sets the value where the timer period ends.
- For up-counting:
 - Counts (CNT) from 0 to ARR
 - Then an update event occurs & the counter restarts

Capture/Compare Register (CCR)

- TIM2 has several (CCR1, CCR2, ...)
- For A4, TIM2 is in Compare mode

When `TIM2->CNT = TIM2->CCR1` the timer can set a compare flag and request an interrupt.

Timer frequency = 4 MHz

Timer Tick = $0.25 \mu\text{s}$

ARR = 799

CCR1 = 199

For a 25% duty cycle, the pin is high for:

$$0.25 \times (200 \mu\text{s}) = 50 \mu\text{s}$$

and low for remaining period. $150 \mu\text{s}$ @ $0.25 \mu\text{s}/\text{tick}$

$$\frac{50 \mu\text{s}}{0.25 \mu\text{s}} = 200 \text{ ticks or counts}$$

$$\therefore \text{CCR1} = 199$$

- The timer counter frequency

$$f_{\text{CNT}} = \frac{f_{\text{TIM2}}}{\text{PSC} + 1}$$

★ We want counter to stay @ full 4 MHz

b. Screen captures Part A – 5 kHz with 25% duty cycle square wave

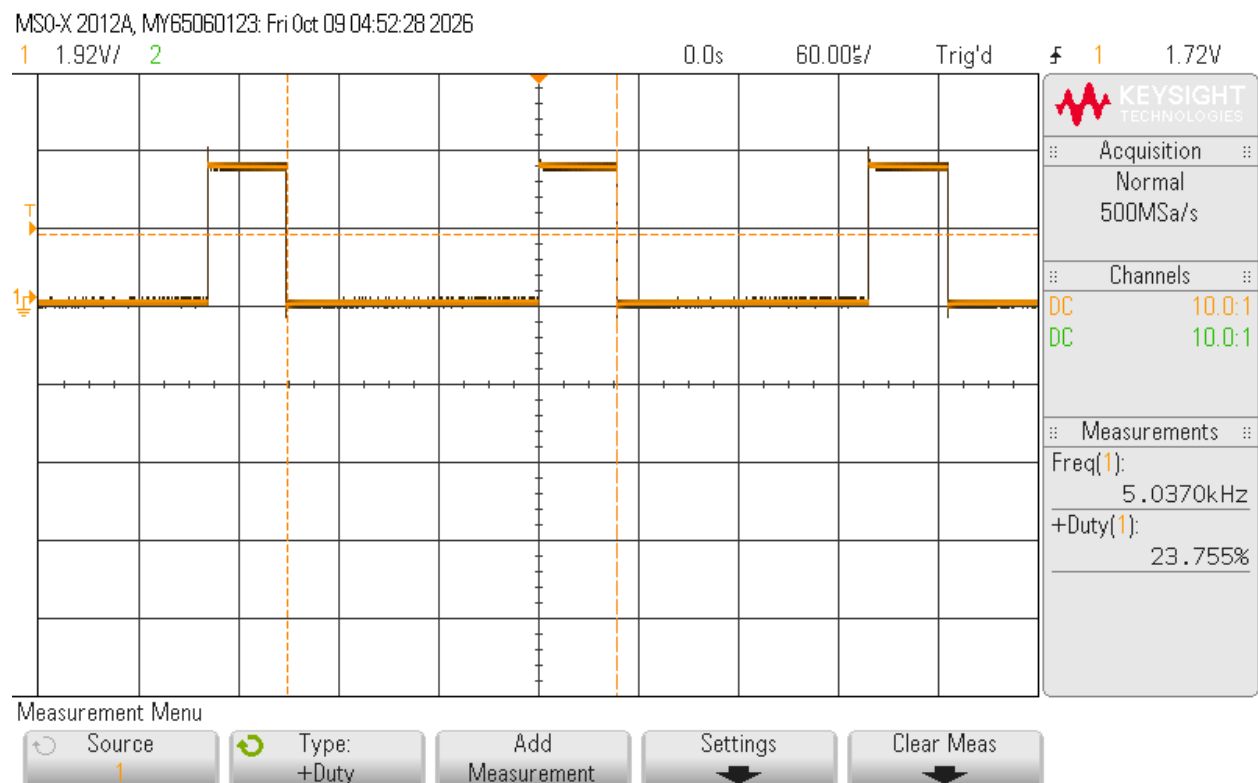


Figure 1: Oscilloscope capture

c. Source Code

main.c

```
1  #include "main.h"
2  void SystemClock_Config(void);
3  int main(void)
4  {
5      HAL_Init();
6
7      // Enable GPIOA peripheral clock FIRST
8      RCC->AHB2ENR |= (RCC_AHB2ENR_GPIOAEN);
9
10     //Setting PA1 up as an output
11     GPIOA->MODER &= ~(GPIO_MODER_MODE1);
12     GPIOA->MODER |= (1U << GPIO_MODER_MODE1_Pos);
13
14     //PA1 is low initially
15     GPIOA->BRR = (1U << 1);
16
17     //Code from lab manual
18     //Enable MCO, select MSI (4 Mhz source)
19     RCC->CFGR = ((RCC->CFGR & ~(RCC_CFGR_MCOSEL)) | (RCC_CFGR_MCOSEL_0));
20
21     // Configure MCO output on PA8
22     GPIOA->MODER &= ~(GPIO_MODER_MODE8); // alternate function mode
23     GPIOA->MODER |= (2 << GPIO_MODER_MODE8_Pos);
24     GPIOA->OTYPER &= ~(GPIO_OTYPER_OT8); // Push-pull output
25     GPIOA->PUPDR &= ~(GPIO_PUPDR_PUPD8); // no resistor
26     GPIOA->OSPEEDR |= (GPIO_OSPEEDR_OSPEED8); // high speed
27     GPIOA->AFR[1] &= ~(GPIO_AFRH_AFSEL8); // select MCO function
28
29     //Enables TIM2 timer at bit 0
30     RCC->APB1ENR1 |= RCC_APB1ENR1_TIM2EN;
31
32     //These values create a 5 kHz with a 25% duty cycle square wave
33     //using a 4 Mhz input clock
34     TIM2->PSC = 0;
35     TIM2->ARR = 799;
36     TIM2->CCR1 = 199;
37
38     //This enables both the update interrupt source and
39     //the channel-1 compare interrupt source.
40     TIM2->DIER |= TIM_DIER_UIE | TIM_DIER_CC1IE;
41
42     //This enables the TIM2 interrupt at the NVIC level
43     NVIC_EnableIRQ(TIM2_IRQn);
44
45     //This enables the timer to start counting
46     //TIM2->CNT starts changing automatically
47     TIM2->CR1 |= TIM_CR1_CEN;
48
49     while (1)
50     {
51     }
52 }
53
54
55 void TIM2_IRQHandler(void)
56 {
57
58     if (TIM2->SR & TIM_SR_CC1IF)
59     {
60         GPIOA->BRR = (1U << 1); // PA1 LOW
61         TIM2->SR = ~TIM_SR_CC1IF; // clear CC1IF here
62     }
63 }
```

```

54         if(TIM2->SR & TIM_SR_UIF)
55         {
56             GPIOA->BSRR = (1U << 1);    // PA1 HIGH
57             TIM2->SR = ~TIM_SR_UIF; //clear UIF here
58         }
59     }
60 }
61
62 /**
63  * @brief System Clock Configuration
64  * @retval None
65  */
66 void SystemClock_Config(void)
67 {
68     RCC_OscInitTypeDef RCC_OscInitStruct = {0};
69     RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
70
71     /** Configure the main internal regulator output voltage
72     */
73     if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
74     {
75         Error_Handler();
76     }
77
78     /** Initializes the RCC Oscillators according to the specified parameters
79     * in the RCC_OscInitTypeDef structure.
80     */
81     RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
82     RCC_OscInitStruct.HSISState = RCC_HSI_ON;
83     RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
84     RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
85     RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
86     RCC_OscInitStruct.PLL.PLLM = 1;
87     RCC_OscInitStruct.PLL.PLLN = 10;
88     RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
89     RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
90     RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
91     if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
92     {
93         Error_Handler();
94     }
95
96     /** Initializes the CPU, AHB and APB buses clocks
97     */
98     RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSClk
99                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
100    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
101    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
102    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
103    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;
104
105    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
106    {
107        Error_Handler();
108    }
109 }
110
111 /**
112  * @brief This function is executed in case of error occurrence.
113  * @retval None
114  */
115 void Error_Handler(void)
116 {
117     /* USER CODE BEGIN Error_Handler_Debug */
118     /* User can add his own implementation to report the HAL error return state */

```

```

182     __disable_irq();
183     while (1)
184     {
185     }
186     /* USER CODE END Error_Handler_Debug */
187 }
188 #ifndef USE_FULL_ASSERT
189 /**
190  * @brief Reports the name of the source file and the source line number
191  *        where the assert_param error has occurred.
192  * @param file: pointer to the source file name
193  * @param line: assert_param error line source number
194  * @retval None
195  */
196 void assert_failed(uint8_t *file, uint32_t line)
197 {
198     /* USER CODE BEGIN 6 */
199     /* User can add his own implementation to report the file name and line number,
200        ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
201     /* USER CODE END 6 */
202 }
203 #endif /* USE_FULL_ASSERT */

```

Listing 1: main.c